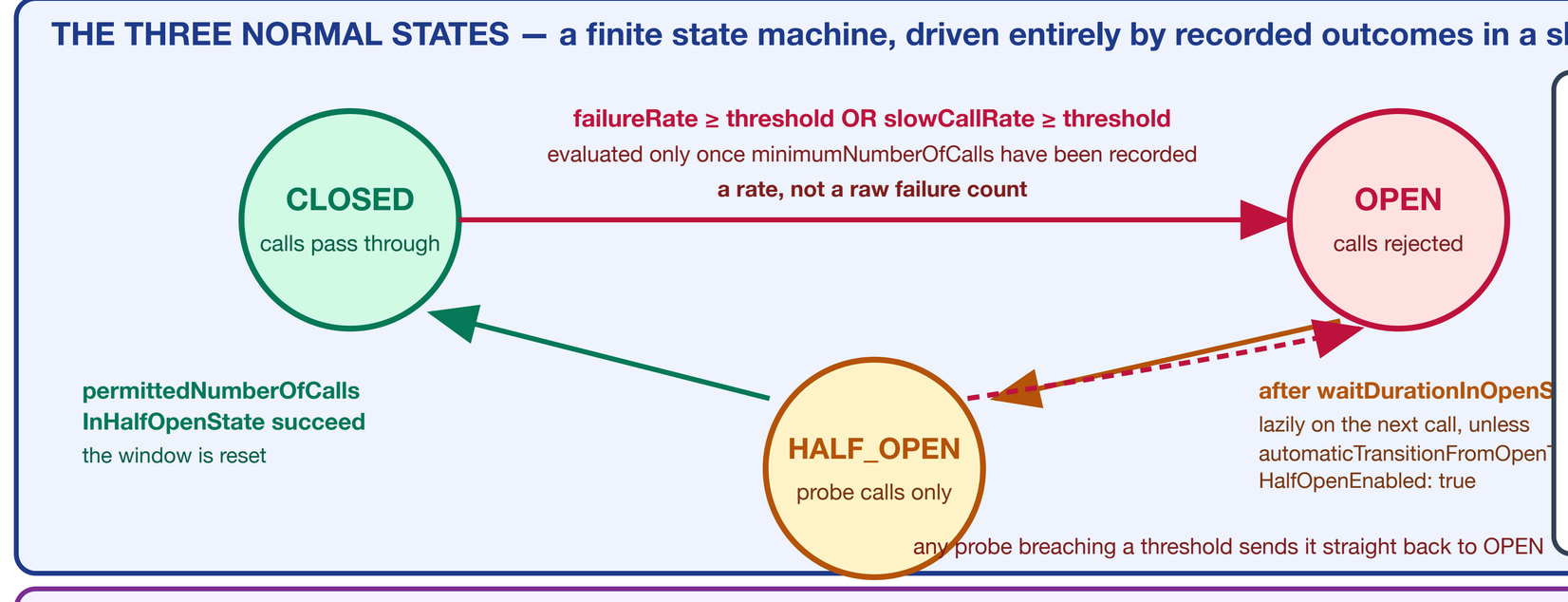
CIRCUIT BREAKER FRAMEWORK IN JAVA — RESILIENCE4J V1

Spring Boot 3.x · resilience4j-spring-boot3 · Java 17+
state machine · sliding windows · aspect order · fallbacks · metrics · testing · the traps that bite in production

1 · STATE MACHINE, THE OPEN DECISION, AND THE RESILIENCE STACK

six states, not three · the circuit opens on a **rate**, never on a count · and the decorator order is fixed unless you change it

THE THREE NORMAL STATES — a finite state machine, driven entirely by recorded outcomes in a sliding window



THE THREE SPECIAL STATES — operational controls, and the ones most cheat sheets omit

FORCED_OPEN	DISABLED	METRICS_ONLY
always denies access your kill switch for a bad dependency	always allows access bypass without removing the annotation	records, never opens shadow mode for tuning thresholds

In FORCED_OPEN and DISABLED no events are generated and no metrics are recorded, apart from the state transition itself. METRICS_ONLY is the exception: it behaves like CLOSED, records everything, and simply never opens — which is what makes it safe for tuning.

Roll out a new breaker in METRICS_ONLY first. Watch the failure rate it *would* have tripped on for a week, then set thresholds from that data and switch to CLOSED. Choosing thresholds up front is guesswork, and a breaker that opens on healthy traffic causes the outage it was added to prevent.

HOW THE OPEN DECISION IS ACTUALLY MADE — the part that determines whether your configuration works

1 · Record the outcome
Every call lands in the sliding window as success, failure, or slow — and slow counts even when it succeeded.
ignoreExceptions are recorded as neither. They do not enter the window at all.

2 · Check the gate
Are there at least `minimumNumberOfCalls` recorded in the window?
If not, the breaker cannot open — no matter how many calls fail.

3 · Compute the rates
failureRate = failed / recorded
slowCallRate = slow / recorded
Both are **percentages**, both are evaluated, and either one alone is enough to trip the breaker.

4 · Open, and reject with CallNotPermittedException
While OPEN, calls never reach the downstream service. The breaker throws `CallNotPermittedException` immediately — this is the "fail fast".
Your fallback receives that exception, so it can distinguish "the circuit is open" from "the call itself failed" and respond differently to each. Catching it blindly and returning a generic error throws that distinction away.

THE LOW-TRAFFIC BLIND SPOT — the most common reason a breaker "does not work"
With `minimumNumberOfCalls: 20` on an endpoint serving two calls a minute, the downstream service can be completely down for ten minutes before the breaker is even permitted to evaluate. Every one of those calls waits for a timeout.
A COUNT_BASED window makes this worse: the window has no notion of time, so twenty calls spread over an hour are treated exactly like twenty calls in one second, and stale failures keep influencing the decision long after the service recovered.
On low-traffic endpoints use TIME_BASED, and lean on the TimeLimiter — a timeout is what protects you before the breaker can.

STATE IS PER JVM — there is no cluster-wide circuit
Each pod keeps its own window and its own state. With twenty replicas you have twenty independent breakers that open at different moments.
Consequences: **divide** traffic by replica count when sizing `minimumNumberOfCalls`; expect a partial, staggered open; and never read one pod's state as the system's. Aggregate the metric across pods.

DECORATOR ORDER — fixed by default, and it changes the meaning of your configuration. Read it outside-in.

Retry	CircuitBreaker	RateLimiter	TimeLimiter	Bulkhead	your method
outermost re-runs everything inside it	sees every attempt 3 retries = 3 entries in the window	throttles attempts retries consume permits too	per attempt not per request — budget accordingly	innermost closest to the call, caps concurrency	the remote call finally happens here

Retry (CircuitBreaker (RateLimiter (TimeLimiter (Bulkhead (Function)))))
This is the default and it is deliberate. Because the breaker sits *inside* Retry, every retry attempt is recorded separately — so three attempts against a dead service push three failures into the window, and the breaker trips roughly three times sooner than a naive reading of your threshold suggests. Account for it when you size the window.
Once the breaker is OPEN, Retry keeps retrying but every attempt fails instantly with `CallNotPermittedException` — cheap, but still three attempts. Exclude it from retry.

Changing the order
Set it explicitly per aspect. Higher value = higher priority = further out:

```
resilience4j.retry.retryAspectOrder
resilience4j.circuitbreaker.circuitBreakerAspectOrder
resilience4j.ratelimiter.rateLimiterAspectOrder
resilience4j.timelimiter.timeLimiterAspectOrder
resilience4j.bulkhead.bulkheadAspectOrder
```

A timeout *outside* retry is a different system
If you want one deadline for the whole request including all retries, the default order will not give it to you — TimeLimiter is inside Retry, so it bounds each attempt and the total is roughly attempts × timeout + backoff.
Either raise `timeLimiterAspectOrder` above retry's, or enforce the overall deadline upstream at the gateway. Decide which you mean — **most teams say "3s timeout" and ship a 10s worst case.**

WHERE EACH PATTERN BELONGS — every one solves a different failure, and they are not substitutes for one another

Clients — web · mobile · partner APIs · no resilience here
→ **API gateway** — authn · global rate limit · overall request deadline · the only true end-to-end budget
→ **Order Service — resilience applied per downstream dependency, never once per service**: **Bulkhead** (caps concurrency) · **TimeLimiter** (bounds one attempt) · **RateLimiter** (protects the callee) · **CircuitBreaker** (stops the bleeding) · **Retry** (transient only)
→ **Payment Service** — slow or failing · its own breaker per dependency, not shared
→ **Fallback / degradation** — cached value · queue for later · default response · partial page · never a second remote call

ONE BREAKER PER DEPENDENCY — the most common configuration mistake
A breaker named after your own service ("orderService") pools unrelated downstreams into one window. Payments going down then opens the circuit for inventory, and a healthy dependency is cut off by a sick neighbour.
Name the instance after what you are calling — paymentService, inventoryService — and give each its own thresholds.

RETRY IS ONLY SAFE ON IDEMPOTENT OPERATIONS
A timeout does not mean the call did not happen. The request may have been received, processed and the response lost. Retrying a payment in that situation charges the customer twice, and no circuit breaker setting prevents it.
Reads: retry freely. Writes: only with a server-enforced idempotency key. Otherwise do not retry at all.

WHAT EACH PATTERN ACTUALLY PROTECTS — pick by failure mode, not by habit
TimeLimiter bounds a single call so one slow dependency cannot hold a thread. **Retry** converts transient faults into successes, and amplifies load during real outages. **CircuitBreaker** stops calling a dependency that is already failing, protecting *you* from waiting and *them* from being hammered.
Bulkhead caps how much of your thread or connection pool one dependency may consume, so its failure cannot starve the rest of the service. **RateLimiter** protects the *callee* from you — a quota you honour, not a failure you react to.
The failure they exist to prevent is the cascade: a slow dependency exhausts your threads, your service stops responding, and its callers fail in turn. Timeouts alone do not stop that — a timeout still consumes the thread for its full duration.

2 · CONFIGURATION REFERENCE

every property that matters, what it means, and the value that is wrong more often than it is right

```
# Maven — Spring Boot 3.x uses the boot3 starter (Jakarta namespace)
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
# plus spring-boot-starter-aop  — the annotations are AOP-driven
# plus spring-boot-starter-actuator + micrometer-registry-prometheus

# application.yml
resilience4j:
  circuitbreaker:
    instances:
      paymentService:              # name the DEPENDENCY
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 100        # calls (COUNT) / seconds (TIME)
        minimumNumberOfCalls: 20      # gate before any evaluation
        failureRateThreshold: 50      # PERCENT, not a count
        slowCallRateThreshold: 100    # PERCENT
        slowCallDurationThreshold: 2s
        waitDurationInOpenState: 30s
        permittedNumberOfCallsInHalfOpenState: 5
        automaticTransitionFromOpenToHalfOpenEnabled: true
        recordExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
        ignoreExceptions:               # wins over recordExceptions
          - com.example.BusinessException

  retry:
    instances:
      paymentService:
        maxAttempts: 3                  # total, not additional
        waitDuration: 200ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        enableRandomizedWait: true      # jitter — stops retry storms
        randomizedWaitFactor: 0.5
        retryExceptions:
          - java.io.IOException
        ignoreExceptions:
          - io.github.resilience4j.circuitbreaker.CallNotPermittedException

  timelimiter:
    instances:
      paymentService:
        timeoutDuration: 3s
        cancelRunningFuture: true
```

Property	What it means, and the mistake to avoid
`slidingWindowType`	**COUNT_BASED** — the last N *calls*. **TIME_BASED** — the last N *seconds*. They are not interchangeable, and the size unit changes meaning with the type.
`slidingWindowSize`	Calls under COUNT_BASED; **seconds** under TIME_BASED. Describing it as "number of calls" is only half right and produces badly-tuned time windows.
`minimumNumberOfCalls`	The breaker cannot open below this, however bad things are. Size it against *per-pod* traffic, not total.
`failureRateThreshold`	Percentage of recorded calls that failed. Default 50. A rate — never a raw count.
`slowCallDurationThreshold`	Above this a call counts as slow **even when it succeeds**. This is what catches a degrading dependency before it starts erroring.
`slowCallRateThreshold`	Percentage of slow calls that trips the breaker. Leave at 100 until you have measured, then lower it deliberately.
`waitDurationInOpenState`	How long OPEN lasts before probes are allowed. Too short and you hammer a recovering service; too long and you stay degraded after it healed.
`permittedNumberOfCallsInHalfOpenState`	Probes allowed in HALF_OPEN. All must succeed to close. Keep it small — these calls hit a service you believe is broken.
`automaticTransitionFromOpenToHalfOpenEnabled`	**false** (default): the move to HALF_OPEN happens lazily, on the next incoming call — so a completely idle breaker stays OPEN indefinitely. **true**: a scheduler thread does it on time.
`recordExceptions`	Only these count as failures. Everything else is a success. Setting it and forgetting one exception type silently disables the breaker for that path.
`ignoreExceptions`	Neither success nor failure — excluded from the window entirely. **Takes precedence** over recordExceptions. This is where 4xx business errors belong.
`maxAttempts`	Total attempts including the first. 3 means one call plus two retries, not four calls.
`enableRandomizedWait`	Jitter. Without it, every instance retries on the same schedule and you rebuild the thundering herd you were avoiding.
`timeoutDuration`	Bounds **one attempt** under the default aspect order, not the whole retried request.

3 · THE SIX STATES

three normal, three operational

State	Behaviour
CLOSED	Calls pass through. Outcomes are recorded in the sliding window. The normal, healthy state.
OPEN	Calls are rejected immediately with `CallNotPermittedException`. Nothing reaches the downstream service. This is the fail-fast that protects both sides.
HALF_OPEN	A limited number of probe calls are permitted. All succeed → CLOSED and the window resets. Any breach → straight back to OPEN.
FORCED_OPEN	Always denies. No events, no metrics beyond the transition. Your operational kill switch.
DISABLED	Always allows. No events, no metrics beyond the transition. Bypass without a code change.
METRICS_ONLY	Behaves like CLOSED and records everything, but **never opens**, whatever the thresholds say.

METRICS_ONLY is the one worth learning. It lets you run a new breaker against real production traffic and observe the failure rate it *would* have tripped on, without any risk of it tripping. Tune from that data, then switch to CLOSED. Guessing thresholds up front is how breakers end up causing outages instead of containing them.

Driving state from code

```java
CircuitBreaker cb = registry.circuitBreaker("paymentService");

cb.transitionToForcedOpenState();   // kill switch on
cb.transitionToClosedState();       // back to normal
cb.transitionToDisabledState();     // bypass entirely

State s = cb.getState();
Metrics m = cb.getMetrics();
float rate = m.getFailureRate();   // -1 until the gate is met
```

What triggers each transition

Transition	Trigger, and what happens to the window
CLOSED → OPEN	A rate threshold is breached with the gate met. The window is retained but no longer consulted while open.
OPEN → HALF_OPEN	`waitDurationInOpenState` elapses — on the next call, or on a scheduler thread if automatic transition is enabled.
HALF_OPEN → CLOSED	The permitted probe calls all complete within thresholds. **The window is reset**, so the breaker starts clean rather than re-tripping on old failures.
HALF_OPEN → OPEN	Any probe breaches a threshold. The wait duration starts again from zero.
any → FORCED_OPEN any → DISABLED	Only through the API or configuration. Never automatic — these are operator decisions.
reset()	Returns to CLOSED and clears all recorded outcomes and metrics.

HALF_OPEN uses its own small window. It is not the main sliding window — it admits exactly `permittedNumberOfCallsInHalfOpenState` calls and judges on those alone. Setting it high means sending a lot of traffic at a service you already believe is broken.

A returned failure rate of −1 is not an error. It means fewer than `minimumNumberOfCalls` have been recorded, so no rate exists yet. Alerting on "failure rate > 0" without excluding −1 produces a permanently firing alert on every low-traffic endpoint.

4 · THE SLIDING WINDOW

choose deliberately

	COUNT_BASED	TIME_BASED
Size unit	The last N calls	The last N **seconds**
Memory	Fixed ring of N outcomes	Partial aggregates per second
Under high traffic	Reacts fast; the window covers a short real interval	Aggregates a lot of calls; smooth and stable
Under low traffic	**Poor.** The window has no notion of time, so failures from an hour ago still count	**Correct.** Old outcomes age out whether or not new calls arrive
Use when	Steady, high call volume	Bursty or low volume — and most real endpoints

The trap in COUNT_BASED. The window is a ring of the last N outcomes with no expiry. On an endpoint doing a handful of calls a minute, a burst of failures at 09:00 is still in the window at 10:00 and can trip the breaker on a service that recovered fifty minutes ago. Time-based windows do not have this failure mode.

Sizing that holds up

- **Start from per-pod traffic.** Twenty replicas sharing 1,000 rpm see 50 rpm each — the breaker only ever sees its own slice.
- **`minimumNumberOfCalls` ≤ `slidingWindowSize`**, and both reachable within a window under normal load. If the gate cannot be met, the breaker is decorative.
- **Window long enough to be statistically meaningful**, short enough to react — tens of seconds is a common landing point, not minutes.
- **Set `slowCallDurationThreshold` from your p99**, not from the timeout. Slow calls are the early warning; errors are the late one.

A worked example

```
# 1,200 rpm to paymentService, 20 pods
per-pod rate        = 1200 / 20   = 60 rpm   = 1 call/sec

# TIME_BASED window of 60s therefore holds ~60 calls per pod
slidingWindowType: TIME_BASED
slidingWindowSize: 60        # SECONDS, not calls
minimumNumberOfCalls: 20     # met in ~20s per pod — reachable

# with retry outside the breaker, a dead dependency
# contributes ~3 failures per request, so the gate is
# met roughly 3x faster during a real outage

# the same config as COUNT_BASED would be wrong:
# slidingWindowSize: 60 = the last 60 CALLS = 60 seconds
# at peak, but a full hour at 1 rpm overnight
```

Rule of thumb. If you cannot state how many calls per minute one pod makes to this dependency, you cannot size the window — and every value in the configuration is a guess.

5 · WORKING CODE

the synchronous form, the asynchronous form, and the annotation combination that silently does nothing

The combination that does not work. `@TimeLimiter` requires the method to return a `CompletionStage`. Placed on a method returning a plain value it cannot bound anything — the blocking work is already finished before the timeout could be scheduled. The application starts, the annotation is present, and no timeout is ever applied.

```java
// DOES NOT WORK — TimeLimiter has nothing to bound
@CircuitBreaker(name = "paymentService", fallbackMethod = "fb")
@TimeLimiter(name = "paymentService")
@Retry(name = "paymentService")
public PaymentResponse pay(PaymentRequest req) {
    return client.pay(req);        // blocking
}

// ALSO A NO-OP — the blocking call runs before the future exists
return CompletableFuture.completedFuture(client.pay(req));
```

Synchronous — drop the TimeLimiter, set the client timeout

```java
@Service
public class PaymentClient {

    private final PaymentServiceClient client;

    @CircuitBreaker(name = "paymentService", fallbackMethod = "fb")
    @Retry(name = "paymentService")
    public PaymentResponse pay(PaymentRequest req) {
        return client.pay(req);
    }

    // most specific match wins — the breaker being open is
    // a different situation from the call having failed
    private PaymentResponse fb(PaymentRequest r, CallNotPermittedException e) {
        return PaymentResponse.unavailable("circuit open");
    }

    private PaymentResponse fb(PaymentRequest r, Throwable t) {
        return PaymentResponse.degraded();
    }
}
```

Without a TimeLimiter, the timeout must come from the client. Set connect and read timeouts on the `RestClient`, `WebClient` or HTTP client itself. A synchronous call with no client timeout can block a request thread indefinitely, and the breaker will not save you — it only reacts *after* calls complete or fail.

Asynchronous — the only form where `@TimeLimiter` works

```java
@CircuitBreaker(name = "paymentService", fallbackMethod = "fb")
@Retry(name = "paymentService")
@TimeLimiter(name = "paymentService")
public CompletableFuture<PaymentResponse> pay(PaymentRequest req) {
    // supplyAsync, so the work runs on another thread and
    // the TimeLimiter can actually interrupt the wait
    return CompletableFuture.supplyAsync(() -> client.pay(req), ioPool);
}

// fallback must return the same type
private CompletableFuture<PaymentResponse> fb(PaymentRequest r, Throwable t) {
    return CompletableFuture.completedFuture(PaymentResponse.degraded());
}
```

`cancelRunningFuture: true` does not stop the work. It completes your future exceptionally, but the thread doing the blocking I/O keeps running until the underlying socket timeout fires. Without a client-level read timeout you leak the thread even though the caller gave up — which is exactly the pool exhaustion the bulkhead exists to prevent.

Fallback rules, in order of how often they are broken

Same class	Resilience4j resolves the method on the same bean. A fallback in a helper class is never found.
Same return type	Including the wrapper — `CompletableFuture<T>` in, `CompletableFuture<T>` out.
Same args + Throwable	The exception parameter is last. Overload it by exception type; the most specific match is used.
Fails at runtime	A signature mismatch is not a compile error. It surfaces as an exception the first time the fallback is needed — in production, during an incident.
Must not call out	A fallback that makes another remote call has no protection of its own and fails when everything else is already failing.
Must be fast	It runs on the caller's thread while the system is degraded. Cache lookups and constants only.

Self-invocation does not work. These annotations are AOP proxies. Calling `this.pay(...)` from another method in the same class bypasses the proxy entirely, so no breaker, no retry, no fallback. It is the most common reason a correctly-configured breaker never triggers.

6 · COMPOSITION

order changes meaning

The default, outside-in.
`Retry ( CircuitBreaker ( RateLimiter ( TimeLimiter ( Bulkhead ( Function ) ) ) ) )`

What follows from it

Retries are recorded	The breaker is inside Retry, so each attempt is a separate entry in the window. Three attempts against a dead service contribute three failures, and the breaker trips sooner than the threshold alone implies.
Timeout is per attempt	Not per request. Worst case ≈ attempts × timeout + backoff. A "3 second timeout" with 3 attempts is a 9-second-plus request.
Open still costs retries	Retry sits outside, so it keeps retrying while the breaker rejects instantly. Cheap, but pointless — add `CallNotPermittedException` to the retry's `ignoreExceptions`.
Bulkhead is innermost	It caps concurrency on the actual call, which is what you want — but it means rejected-by-bulkhead is seen by the breaker as a failure unless you exclude it.

Changing it

```yaml
# higher value = higher priority = further OUT
resilience4j:
  timelimiter:
    timeLimiterAspectOrder: 4   # now outside retry
  retry:
    retryAspectOrder: 3         # one overall deadline
```

Decide what your timeout means. A per-attempt timeout bounds each try and lets the total grow. A per-request deadline bounds the user's wait and may cut a retry short. Both are defensible; shipping one while believing you have the other is not.

Two bulkheads, and only one of them is the default

Semaphore the default for `@Bulkhead`	Caps concurrent calls with a counter. No extra threads, so the caller's thread still blocks for the duration of the call.
Fixed thread pool `@Bulkhead(type = THREADPOOL)`	Runs the call on its own bounded pool with a queue, so a slow dependency cannot consume request threads. **Requires a `CompletableFuture` return type**, exactly like `@TimeLimiter`.

The safest default is to enforce the overall deadline upstream — at the gateway or the caller — and leave the aspect order alone. It survives configuration drift and applies to every path, including the ones nobody annotated.

7 · FALLBACKS & DEGRADATION

what to return

A fallback is a product decision wearing a technical costume. "Return an empty list" is a choice that someone will see in the UI. Decide it with whoever owns that screen, not in the pull request.

Strategy	When it is right
Cached value	The data changes slowly and stale is better than absent — pricing, catalogue, configuration. Return the age with it.
Default response	A safe neutral exists: empty recommendations, standard shipping estimate, no promotions.
Queue and process later	The write can be deferred and the caller told so. Needs an idempotency key and a way to report the outcome.
Partial response	Render the page without the failing component rather than failing the whole page.
Fail explicitly	Payments, balances, anything where a wrong answer is worse than no answer. **Often the correct choice** — a fallback is not always an improvement.

Distinguish the two failure situations. `CallNotPermittedException` means the circuit is open — the dependency is known bad and you should degrade immediately. Any other throwable means this particular call failed and the dependency may still be healthy. They deserve different messages and often different actions.

Silent fallbacks hide outages. If a fallback returns a plausible response and nothing is emitted, your dashboards stay green through a total dependency failure. Every fallback invocation must increment a metric and be visible on the same dashboard as the breaker state.

Signature, and the overload that earns its keep

```java
// circuit is open — the dependency is known bad
private Quote fb(Request r, CallNotPermittedException e) {
    fallbackCounter.increment("circuit_open");
    return cache.lastKnown(r).orElseGet(Quote::unavailable);
}

// this call failed — the dependency may still be fine
private Quote fb(Request r, Throwable t) {
    fallbackCounter.increment("call_failed");
    return Quote.unavailable();
}
```

Test the fallback path as a first-class scenario. It runs rarely, under stress, and is usually the least-exercised code in the service. Force the breaker open in an integration test and assert on what the customer actually receives.

8 · RETRY

the pattern most likely to cause the outage

Retry amplifies load exactly when the system is least able to take it. Three attempts per request means a struggling dependency receives three times the traffic at the moment it is already failing. The circuit breaker exists partly to stop this.

Before enabling it, answer these

Is it idempotent?	A timeout does not mean the call did not happen — the work may have completed and the response been lost. Retrying a charge in that state bills twice.
Is the fault transient?	Connection reset, timeout, 503 — yes. 400, 401, 404, 422 — never. Retrying a deterministic failure is pure waste.
Is there jitter?	Without randomisation every instance retries in lockstep and you rebuild the thundering herd.
What is the budget?	attempts × timeout + backoff must fit inside the caller's deadline, or you produce a slow failure instead of a fast one.

Configuration that behaves

```yaml
maxAttempts: 3                  # total: 1 call + 2 retries
waitDuration: 200ms
enableExponentialBackoff: true
exponentialBackoffMultiplier: 2   # 200ms, 400ms
enableRandomizedWait: true
randomizedWaitFactor: 0.5
retryExceptions:
  - java.io.IOException
ignoreExceptions:
  - …CallNotPermittedException   # do not retry an open circuit
```

Two or three attempts, not five. Beyond a small number the marginal success rate is negligible and the load amplification is not. If two retries do not fix it, the dependency is not having a blip.

Retries at every layer multiply. Gateway retries × service retries × client retries is 27 requests from three levels of three. Decide which single layer owns retry, and turn it off everywhere else.

9 · OPERATING IT

rollout, tuning and the runtime controls that matter during an incident

Rolling out a new breaker

Step	What you do
1	Set the client-level connect and read timeouts first. Without them nothing else helps, because the breaker only reacts to completed calls.
2	Deploy in **METRICS_ONLY**. It records everything and cannot open.
3	Watch for a full traffic cycle — a week, including the weekly peak and any batch window.
4	Set `slowCallDurationThreshold` from the observed p99, and the rate thresholds from what actually happened during real incidents.
5	Switch to CLOSED. Alert on state transitions from day one.
6	Run a game day: make the dependency fail and confirm the breaker opens, the fallback fires and the dashboards show it.

Thresholds guessed at design time are usually wrong in both directions. Too sensitive and the breaker opens during normal variance, turning a slow dependency into a hard outage. Too lax and it never opens, so the cascade it was added to prevent happens anyway with extra configuration on top.

Actuator surface

```
# health, state and event history
GET /actuator/health              # circuitBreakers section
GET /actuator/circuitbreakers     # current state per instance
GET /actuator/circuitbreakerevents
GET /actuator/circuitbreakerevents/{name}/{type}
GET /actuator/metrics/resilience4j.circuitbreaker.calls

# the event stream is a ring buffer, not a log —
# size it, and ship transitions to your log pipeline too
resilience4j.circuitbreaker.configs.default.eventConsumerBufferSize: 100
```

Reacting to transitions in code

```java
cb.getEventPublisher()
  .onStateTransition(e -> log.warn("{} {} -> {}",
        e.getCircuitBreakerName(),
        e.getStateTransition().getFromState(),
        e.getStateTransition().getToState()))
  .onCallNotPermitted(e -> rejected.increment())
  .onError(e -> log.debug("failed after {}", e.getElapsedDuration()));
```

Emit a transition event, not just a gauge. A breaker that opens and closes between two metric scrapes is invisible on a dashboard but very visible to customers. The event stream catches what sampling misses.

During an incident

Dependency known bad	`transitionToForcedOpenState()` — stop calling it entirely, serve fallbacks, and stop the noise while you work.
Breaker misfiring	`transitionToDisabledState()` — bypass it without a deploy, then fix the thresholds properly afterwards.
Dependency recovered	`transitionToClosedState()` — resets the window rather than waiting out the open period.

Expose these deliberately. A guarded admin endpoint that can force a named breaker open is one of the highest-value operational tools you can ship — and one of the cheapest. Authorise it properly and audit every use; it can also take a healthy dependency offline.

Remember the state is per pod. Forcing a breaker open through one instance's endpoint affects only that instance. Drive it from configuration or a control plane if you need it fleet-wide, and never conclude "the circuit is closed" from a single pod's response.

Share configuration, override per dependency

```yaml
resilience4j.circuitbreaker:
  configs:
    default:                      # applies to every instance
      slidingWindowType: TIME_BASED
      slidingWindowSize: 60
      failureRateThreshold: 50
  instances:
    paymentService:
      baseConfig: default          # inherit, then override
      waitDurationInOpenState: 60s  # payments recover slowly
```

Alert on the transition, not the state gauge. Alerting on `state == OPEN` misses fast flapping and fires continuously during a long outage. Alert on the rate of transitions, and separately on sustained time in OPEN.

10 · METRICS

the real names

Resilience4j publishes Micrometer metrics automatically. Micrometer uses dot notation internally; the Prometheus registry converts dots to underscores, which is why the exported names differ from the ones in the Java API.

```
# exported to Prometheus
resilience4j_circuitbreaker_state
  tags: name, state    # one series per state, value 0/1

resilience4j_circuitbreaker_calls_seconds_count
resilience4j_circuitbreaker_calls_seconds_sum
resilience4j_circuitbreaker_calls_seconds_max
  tags: name, kind=successful|failed  # it is a TIMER,
  # so you get latency and counts from the same metric

resilience4j_circuitbreaker_not_permitted_calls_total
  tags: name, kind=not_permitted   # rejected while OPEN

resilience4j_circuitbreaker_buffered_calls
  tags: name, kind    # what is in the window right now

resilience4j_circuitbreaker_slow_calls
resilience4j_circuitbreaker_failure_rate
resilience4j_circuitbreaker_slow_call_rate
```

There is no `..._failures_total` counter. Failures come from the `calls` timer filtered by `kind="failed"`. Dashboards written against a guessed metric name render empty panels and are usually only discovered during the incident they were built for — confirm every name against `/actuator/prometheus` before you rely on it.

Four panels worth having

- **State over time**, per instance — as a discrete band, so flapping is visible rather than averaged away.
- **Failure rate against the configured threshold**, so you can see how close to tripping you normally run.
- **Not-permitted call rate** — this is customer impact. It counts requests that never reached the dependency.
- **Fallback invocations**, from your own counter. Resilience4j does not publish one, and without it a silent fallback keeps the dashboard green.

Queries that answer the real questions

```
# is any instance open right now?
max by(name) (resilience4j_circuitbreaker_state{state="open"})

# customer impact: calls rejected without reaching the service
sum by(name) (rate(
  resilience4j_circuitbreaker_not_permitted_calls_total[5m]))

# failure ratio — from the calls TIMER, not a failures counter
sum(rate(resilience4j_circuitbreaker_calls_seconds_count{
      kind="failed"}[5m])) by(name)
/ sum(rate(resilience4j_circuitbreaker_calls_seconds_count[5m])) by(name)
```

Aggregate across pods, and also break down by pod. The aggregate tells you whether the dependency is failing; the breakdown tells you whether it is one bad instance — which is a completely different incident.

11 · TESTING

it is untested until you have opened it

An unopened breaker is unverified configuration. Nearly every failure mode on this sheet — a missing fallback signature, a no-op TimeLimiter, a gate that can never be met, self-invocation bypassing the proxy — is invisible until the breaker actually trips.

Drive the state directly

```java
@Test
void servesFallbackWhenCircuitIsOpen() {
    CircuitBreaker cb = registry.circuitBreaker("paymentService");
    cb.transitionToOpenState();

    PaymentResponse r = paymentClient.pay(request);

    assertThat(r.status()).isEqualTo("UNAVAILABLE");
    assertThat(cb.getState()).isEqualTo(State.OPEN);
}
```

A test for each of these

- **Fallback signature resolves** — for every overload, including the `CallNotPermittedException` one. A mismatch is a runtime failure, not a compile error.
- **The breaker actually opens** under the configured rate, using a stub that fails deterministically.
- **HALF_OPEN closes on success and reopens on failure** — the transition most often left unverified.
- **The timeout fires**, with a stub that sleeps past the threshold. This is what catches a no-op TimeLimiter.
- **Retry does not multiply a non-idempotent write** — assert the number of calls the stub received.
- **Business exceptions do not open the circuit** — an `ignoreExceptions` regression is silent otherwise.

Use a separate test profile with small numbers. `minimumNumberOfCalls: 2`, a short wait duration, automatic transition enabled. Testing against production values makes tests slow, flaky, and quietly disabled.

Then verify it once in a real environment. A game day against a genuinely unavailable dependency is the only thing that exercises the timeouts, the pools, the dashboards and the alerts together — which is the combination that actually fails.

12 · TRAPS THAT BITE IN PRODUCTION

configurations that look correct, compile, deploy cleanly — and do nothing, or the wrong thing

Trap	What actually happens	Fix
`@TimeLimiter` on a sync method	It requires a `CompletionStage`. On a method returning a plain value there is nothing to bound, and no timeout is applied at all.	Return `CompletableFuture` via `supplyAsync`, or drop the annotation and set client timeouts.
`completedFuture` wrapper	Looks async, is not. The blocking call runs to completion before the future exists, so the TimeLimiter has already lost.	`supplyAsync(() -> call(), pool)`, on a pool you control.
Self-invocation	AOP proxies are bypassed on internal calls, so no breaker, retry or fallback applies. Everything looks configured and nothing is active.	Call through an injected bean, or restructure so the entry point is external.
Assuming a failure count	`failureRateThreshold` is a percentage. Setting it to 5 expecting "5 failures" opens the circuit at a 5% failure rate.	Read it as a rate, and set the count expectation via the window and the gate.
Unreachable gate	On low traffic `minimumNumberOfCalls` is never met, so the breaker cannot open however bad things get.	Size against per-pod traffic, use TIME_BASED, and rely on timeouts.
Breaker named after your own service	All downstreams share one window, so one sick dependency opens the circuit for healthy ones.	One instance per dependency, named after the callee.
Business errors recorded	Validation failures and 404s count as failures, so a burst of bad client input trips a breaker on a perfectly healthy service.	Put them in `ignoreExceptions`, which takes precedence.
Fallback signature mismatch	Not a compile error. It fails the first time the fallback is needed — during the incident it was written for.	A test per overload, asserting the returned value.

Trap	What actually happens	Fix
Fallback makes a remote call	It runs when the system is already degraded, has no protection of its own, and fails or hangs — converting a graceful degradation into a second outage.	Cache, constant or queue only. Nothing over the network.
Silent fallback	A plausible response is returned, no metric moves, and dashboards stay green through a total dependency failure.	Increment a counter on every fallback and alert on its rate.
Retrying an open circuit	Retry sits outside the breaker, so it keeps attempting and each fails instantly. Harmless in cost, but it distorts your retry metrics.	Add `CallNotPermittedException` to the retry's `ignoreExceptions`.
Retrying non-idempotent writes	A timeout does not prove the call did not land. The customer is charged twice and no breaker setting prevents it.	Server-enforced idempotency key, or no retry on writes.
Retries at every layer	Gateway × service × client compounds multiplicatively — three levels of three attempts is 27 requests at a dependency that is already failing.	One layer owns retry. Disable it everywhere else, explicitly.
No client-level timeout	The breaker only reacts to calls that complete or fail. A call that hangs forever holds a thread forever, and the window never fills.	Connect and read timeouts on every HTTP client, always.
Expecting a cluster-wide circuit	State is per JVM. Twenty pods means twenty breakers opening at twenty different moments, and one pod's state tells you nothing about the fleet.	Aggregate metrics across pods; size thresholds per pod.
Dashboards on guessed metric names	`..._failures_total` does not exist. The panel renders empty and nobody notices until an incident.	Copy names from `/actuator/prometheus` on a running instance.
Never opened in a test	Every trap above stays invisible until the breaker trips for the first time in production.	`transitionToOpenState()` in tests, plus a game day.

13 · PRODUCTION READINESS CHECKLIST

confirm before a breaker sits between your service and a dependency that pays the bills

- ☐ **Connect and read timeouts set on the HTTP client** — before any annotation
- ☐ **One breaker instance per dependency**, named after the callee
- ☐ **Thresholds derived from measured traffic**, not chosen at design time
- ☐ **Rolled out in METRICS_ONLY first** for a full traffic cycle
- ☐ **Window sized against per-pod traffic**, not fleet totals
- ☐ **`minimumNumberOfCalls` reachable** under normal load
- ☐ **TIME_BASED chosen** for low-traffic or bursty endpoints
- ☐ **`slowCallDurationThreshold` set from p99**, not from the timeout
- ☐ **`failureRateThreshold` understood as a percentage**
- ☐ **Business and validation errors in `ignoreExceptions`**
- ☐ **`recordExceptions` reviewed** — every genuine failure type is listed
- ☐ **`automaticTransitionFromOpenToHalfOpenEnabled` chosen deliberately**
- ☐ **`@TimeLimiter` only on `CompletionStage`-returning methods**
- ☐ **No `completedFuture` wrapper** around a blocking call
- ☐ **No self-invocation** — every annotated method is entered through the proxy
- ☐ **Aspect order understood**; timeout meaning stated as per-attempt or per-request
- ☐ **Overall request deadline enforced somewhere**, usually upstream
- ☐ **`CallNotPermittedException` excluded from retry**
- ☐ **Retry only on idempotent operations**, or with a server-enforced key
- ☐ **Exactly one layer owns retry**; disabled explicitly elsewhere
- ☐ **Jitter enabled** on retry backoff
- ☐ **Retry budget fits** inside the caller's deadline
- ☐ **Fallback in the same class**, same return type, args + Throwable
- ☐ **Fallback overloaded** for `CallNotPermittedException` separately
- ☐ **Fallback makes no remote call** and is fast
- ☐ **Fallback content agreed** with whoever owns the user-facing behaviour
- ☐ **Every fallback increments a counter** — no silent degradation
- ☐ **Bulkhead sized** so one dependency cannot exhaust the pool
- ☐ **Metric names copied from `/actuator/prometheus`**, not guessed
- ☐ **Dashboard shows state, failure rate, not-permitted rate, fallback rate**
- ☐ **Metrics aggregated across pods** and broken down by pod
- ☐ **Alert on state transitions**, not only on the state gauge
- ☐ **Failure rate of −1 excluded** from alerting
- ☐ **State transitions logged**, not only held in the event ring buffer
- ☐ **Test proves the breaker opens** at the configured rate
- ☐ **Test proves HALF_OPEN closes and reopens**
- ☐ **Test proves the timeout fires** — catches a no-op TimeLimiter
- ☐ **Test proves business exceptions do not open** the circuit
- ☐ **Separate test profile** with small windows and short waits
- ☐ **Forced-open kill switch exists**, authorised and audited
- ☐ **Runbook** for open circuit, misfiring breaker, and recovery
- ☐ **Game day run** against a genuinely unavailable dependency

The one that summarises the sheet: **make the dependency fail, and watch what the customer receives.** If you have never seen your own breaker open, you have configuration — not resilience.